

DÉVELOPPEUR WEB APP & WEB MOBILE
RNCP 5

GOUSSET Antoine
BOISARD Gwendal
HOURIEZ Yonas

C28

PROJET “KERNO”

RAPPORT STAGE 5

20
26

Holberton
DEFINE YOUR FUTURE

Table des matières

01 **Introduction**

- 1.1 Présentation générale du projet
- 1.2 Contexte du projet
- 1.3 Objectif du rapport final
- 1.4 État final du MVP

02 **Objectifs initiaux et périmètre du MVP**

- 2.1 Objectif de la section
- 2.2 Objectifs fonctionnels initiaux
- 2.3 Fonctionnalités volontairement exclues
- 2.4 Objectifs techniques initiaux
- 2.5 Objectifs organisationnels initiaux

03 **Résumé du MVP livré**

- 3.1 Objectif de la section
- 3.2 Parcours fournisseur livré
- 3.3 Parcours magasin livré
- 3.4 Fonctionnalités principales livrées
- 3.5 État de déploiement

04 **Comparaison entre les objectifs initiaux et les résultats finaux**

- 4.1 Objectif de la section
- 4.2 Comparaison fonctionnelle globale
- 4.3 Comparaison technique
- 4.4 Comparaison organisationnelle
- 4.5 Écart principal restant : le déploiement
- 4.6 Synthèse de la comparaison

05 **Résultats finaux et livrables produits**

- 5.1 Objectif de la section
- 5.2 Livrable produit principal
- 5.3 Livrables frontend
- 5.4 Livrables backend
- 5.5 Livrables base de données
- 5.6 Livrables API et Swagger
- 5.7 Livrables tests, qualité et sécurité
- 5.8 Livrables documentation
- 5.9 Livrables organisationnels
- 5.10 Livrables de démonstration
- 5.11 Métriques finales des livrables
- 5.12 Synthèse des résultats finaux

Table des matières

06 Difficultés rencontrées et solutions apportées

- 6.1 Objectif de la section
- 6.2 Maîtrise du périmètre MVP
- 6.3 Organisation du suivi projet
- 6.4 Intégration frontend/backend
- 6.5 Gestion des rôles et des accès
- 6.6 Qualité visuelle et cohérence UX
- 6.7 Tests et stabilisation
- 6.8 Sécurité et préparation production
- 6.9 Déploiement non réalisé à ce stade
- 6.10 Synthèse des difficultés

07 Leçons apprises

- 7.1 Objectif de la section
- 7.2 Le cadrage MVP est essentiel
- 7.3 Une documentation initiale aide, mais doit rester vivante
- 7.4 GitHub Project était plus adapté que Trello
- 7.5 Le monolithe modulaire est adapté à un MVP
- 7.6 L'intégration doit être testée en continu
- 7.7 La qualité visuelle prend du temps
- 7.8 La sécurité doit être anticipée, même sur un MVP local
- 7.9 Le déploiement doit être prévu plus tôt
- 7.10 Synthèse des leçons apprises

08 Rétrospective équipe

- 8.1 Objectif de la section
- 8.2 Organisation de l'équipe
- 8.3 Ce qui a bien fonctionné
- 8.4 Difficultés rencontrées en équipe
- 8.5 Axes d'amélioration pour de futurs projets
- 8.6 Bilan de collaboration

09 Conclusion et recommandations futures

- 9.1 Objectif de la section
- 9.2 Bilan général du projet
- 9.3 Résultat final
- 9.4 Limites assumées
- 9.5 Recommandations techniques
- 9.6 Recommandations produit
- 9.7 Prochaines étapes recommandées
- 9.8 Conclusion finale



01

Introduction

1. Introduction

1.1 Présentation générale du projet

KERNO est un projet de marketplace B2B SaaS destiné à connecter des fournisseurs directs ou locaux avec des magasins de détail. L'objectif principal du projet est de simplifier la découverte de fournisseurs, de rendre les offres de produits plus visibles et de structurer le premier contact commercial entre les deux parties.

Le projet a été mené dans le cadre du Portfolio Project Holberton par une équipe composée d'Antoine Gousset, Gwendal Boisard et Yonas Houriez. Après les phases d'idéation, de cadrage, de documentation technique et de développement, ce rapport final présente les résultats obtenus, les écarts éventuels avec les objectifs initiaux, les difficultés rencontrées, les solutions mises en place et les enseignements tirés de l'ensemble du cycle projet.

Le MVP livré ne cherche pas à couvrir tout le cycle d'achat. Il se concentre volontairement sur une chaîne simple et réaliste : un fournisseur crée son profil, ajoute ses produits, un magasin recherche une offre, consulte les informations disponibles, puis envoie une demande de contact ou de devis. Le fournisseur peut ensuite consulter les demandes reçues et identifier les opportunités commerciales générées par sa visibilité sur la plateforme.

Ce périmètre volontairement maîtrisé permet de valider la proposition de valeur principale du projet sans intégrer prématurément des fonctionnalités complexes comme le paiement, la commande complète, la livraison, la facturation, la messagerie avancée ou les analytics détaillés. Ces éléments restent des pistes d'évolution possibles, mais ne sont pas nécessaires à la validation du MVP.

1.2 Contexte du projet

Le point de départ de KERNO repose sur un constat simple : pour de nombreux magasins de détail, identifier rapidement des fournisseurs pertinents, fiables et adaptés à leurs besoins peut être chronophage. De l'autre côté, les fournisseurs locaux ou directs peuvent manquer de visibilité auprès des magasins susceptibles d'être intéressés par leurs produits.

KERNO répond à ce problème en proposant une plateforme qui centralise les profils fournisseurs, les produits disponibles et les premières demandes commerciales. Le projet ne remplace pas une négociation commerciale complète et ne se positionne pas encore comme une solution transactionnelle. Il intervient en amont, au moment où un magasin cherche une offre et où un fournisseur souhaite être découvert.

Cette orientation a guidé les choix fonctionnels et techniques du MVP. Le projet devait rester assez simple pour être développé dans le cadre d'un Portfolio Project, mais suffisamment structuré pour démontrer une vraie logique produit, une architecture claire et un parcours utilisateur exploitable.

1.3 Objectif du rapport final

Ce rapport final a pour objectif de clôturer le projet en documentant les résultats obtenus et les apprentissages réalisés. Il permet de comparer le MVP livré aux objectifs initiaux, d'identifier les réussites, d'expliquer les limites assumées et de préparer la présentation finale du projet.

Il sert également de support de recul sur l'organisation d'équipe. Le projet KERNO n'a pas seulement consisté à produire du code. Il a nécessité du cadrage, de la priorisation, des choix techniques, des arbitrages UX, une organisation Git, des tests, des corrections, de la documentation et une préparation à la revue manuelle.

Ce rapport montre donc que le projet est allé au bout de son cycle MVP : conception, développement, stabilisation, documentation, démonstration et réflexion finale.

1.4 État final du MVP

À la clôture du Stage 5, le MVP KERNO est terminé pour son périmètre prévu. Les parcours principaux fournisseur et magasin sont disponibles en local. Le fournisseur peut se connecter, gérer son profil, publier ou gérer ses produits et consulter les demandes reçues. Le magasin peut accéder au catalogue, consulter des pages fournisseur ou produit, envoyer une demande structurée et suivre ses demandes envoyées.

Le projet est actuellement exploitable en environnement local. Il n'est pas encore déployé en production. Cette limite est assumée : le déploiement est identifié comme une étape future, liée à la poursuite entrepreneuriale du projet. Le MVP actuel permet de démontrer la valeur principale, de valider les parcours critiques et de préparer les prochaines étapes techniques.

KERNO doit donc être présenté comme un MVP fonctionnel et documenté, mais pas comme une marketplace complète déjà prête à être exploitée commercialement en production.



02

Objectifs initiaux et
périmètre du MVP

2. Objectifs initiaux et périmètre du MVP

2.1 Objectif de la section

Cette section présente les objectifs initiaux du projet KERNO et rappelle le périmètre défini avant le développement du MVP. Elle permet de comparer la vision de départ avec le résultat final livré à la clôture du Stage 5.

Lors du Stage 3, le projet KERNO avait été cadré comme une marketplace B2B SaaS destinée à connecter des fournisseurs directs ou locaux avec des magasins de détail. L'objectif principal était de simplifier la découverte de fournisseurs, de rendre les offres de produits plus visibles et de structurer le premier contact commercial entre les deux parties.

Le projet ne visait pas à créer une marketplace complète dès la première version. Le MVP devait rester volontairement limité à un parcours principal simple : un fournisseur crée son profil, ajoute ses produits, un magasin recherche une offre, consulte les informations disponibles, puis envoie une demande de contact ou de devis. Le fournisseur peut ensuite recevoir et consulter cette demande.

2.2 Objectifs fonctionnels initiaux

L'objectif fonctionnel principal était de valider la mise en relation entre deux types d'utilisateurs : les fournisseurs et les magasins. Le fournisseur devait pouvoir construire sa visibilité sur la plateforme grâce à un profil professionnel et à un catalogue de produits. Le magasin devait pouvoir rechercher des fournisseurs ou des produits, consulter des informations utiles, puis envoyer une demande structurée.

Le MVP devait donc couvrir les fonctionnalités indispensables suivantes : inscription et connexion, distinction des rôles fournisseur et magasin, création de profils, publication de produits, catalogue consultable, pages de détail, envoi de demandes et consultation des demandes reçues ou envoyées.

La priorisation MoSCoW définie au Stage 3 avait permis de séparer les fonctionnalités essentielles des améliorations secondaires. Les éléments Must Have représentaient le cœur du MVP : inscription, profils, produits, recherche, pages de détail, demande de contact et consultation des demandes. Sans ces éléments, la proposition de valeur principale ne pouvait pas être validée.

Les fonctionnalités Should Have, comme les tableaux de bord, le suivi simple des demandes et la progression des profils, devaient améliorer la lisibilité du produit sans devenir bloquantes. Les fonctionnalités Could Have apportaient du confort d'usage, mais ne devaient pas ralentir la livraison du MVP.

2.3 Fonctionnalités volontairement exclues

Dès la phase de cadrage, plusieurs fonctionnalités ont été exclues du MVP afin de maintenir un périmètre réaliste. Le projet ne devait pas intégrer le paiement en ligne, les commandes complètes, la livraison, la facturation, une messagerie avancée, des avis publics, des analytics détaillés ou une gestion complexe des abonnements.

Ce choix était essentiel pour éviter la dérive du périmètre. KERN0 devait d'abord démontrer qu'une mise en relation structurée entre fournisseurs et magasins pouvait fonctionner. Les fonctionnalités transactionnelles ou avancées pourront être étudiées plus tard, mais elles n'étaient pas nécessaires pour valider la première version.

Cette décision reste cohérente avec l'état final du projet. À la clôture du Stage 5, KERN0 est un MVP fonctionnel centré sur la découverte, la visibilité produit et la demande de contact. Il ne doit pas être présenté comme une plateforme e-commerce complète, mais comme une base produit solide pour une future évolution entrepreneuriale.

2.4 Objectifs techniques initiaux

Les objectifs techniques étaient de construire une application web claire, maintenable et testable. Le Stage 3 avait défini une stack volontairement classique : React, JavaScript, Vite et Tailwind CSS pour le frontend ; Node.js, Express et JavaScript pour le backend ; PostgreSQL et Prisma ORM pour la base de données ; une API REST documentée avec OpenAPI / Swagger ; et une architecture backend en monolithe modulaire.

Ces choix répondaient à plusieurs contraintes. Le projet devait être réalisable par une petite équipe, compréhensible lors d'une revue technique, suffisamment structuré pour évoluer, mais sans complexité excessive. Le monolithe modulaire permettait de garder une seule API backend tout en séparant clairement les domaines fonctionnels : authentification, utilisateurs, fournisseurs, magasins, produits, catégories et demandes.

L'objectif n'était donc pas de construire l'architecture la plus complexe possible, mais de produire une base fiable, lisible et adaptée au niveau de maturité d'un MVP Holberton.

2.5 Objectifs organisationnels initiaux

Au-delà du produit lui-même, le projet devait aussi démontrer une capacité d'organisation. Le développement devait s'appuyer sur Git, GitHub, des branches, des issues, des pull requests, une documentation claire, des tests et une préparation à la revue manuelle.

Lors du cadrage initial, l'équipe avait envisagé l'utilisation de Trello pour le suivi des tâches. Pendant le développement, ce choix a évolué vers l'utilisation de GitHub Project, plus cohérent avec le dépôt GitHub, les issues, les branches et les pull requests. Ce changement a permis de centraliser le suivi projet et le développement dans un même environnement, en reliant plus facilement les tâches aux évolutions réelles du code.

Cette organisation permet de montrer que KERNO n'a pas seulement été développé comme une suite de fichiers de code, mais comme un projet structuré, documenté et préparé pour être expliqué à des évaluateurs ou à de futurs contributeurs.



03

Résumé du MVP livré

3. Résumé du MVP livré

3.1 Objectif de la section

Cette section présente l'état final du MVP KERNO à la clôture du Stage 5. Elle décrit les fonctionnalités livrées, les parcours utilisateurs disponibles, les éléments techniques réalisés et les limites assumées de cette première version.

Le MVP livré respecte le périmètre défini lors des étapes précédentes. Il permet de démontrer la chaîne de valeur principale du projet : un fournisseur peut créer sa présence sur la plateforme, publier des produits, être visible dans un catalogue, puis recevoir une demande envoyée par un magasin. De son côté, un magasin peut rechercher une offre, consulter les informations disponibles et initier un premier contact commercial structuré.

3.2 Parcours fournisseur livré

Le parcours fournisseur est l'un des deux parcours principaux du MVP. Il permet à un utilisateur ayant le rôle fournisseur de se connecter, d'accéder à son espace dédié, de compléter son profil professionnel, de gérer ses produits et de consulter les demandes reçues.

Le fournisseur dispose d'un tableau de bord qui donne une vue synthétique de son activité. Cette page sert de point d'entrée vers les actions principales : améliorer le profil, ajouter un produit, consulter les produits visibles dans le catalogue et suivre les demandes envoyées par les magasins.

La gestion du profil fournisseur permet de renseigner les informations utiles pour présenter l'entreprise : nom, description, localisation, activité, coordonnées et informations de contact. Ce profil sert ensuite de base aux pages visibles par les magasins. L'objectif est de rendre le fournisseur crédible, identifiable et exploitable dans un contexte B2B.

Le fournisseur peut également créer et gérer des produits. Chaque produit est associé à des informations essentielles comme le nom, la description, la catégorie, le prix ou l'unité de prix, le colisage, l'origine et l'image produit. Ces informations alimentent ensuite le catalogue consulté par les magasins.

Enfin, le fournisseur peut consulter les demandes reçues. Cette fonctionnalité clôture le parcours principal côté fournisseur : la visibilité générée par le profil et les produits se transforme en opportunités commerciales concrètes.

3.3 Parcours magasin livré

Le parcours magasin constitue le second parcours principal du MVP. Il permet à un utilisateur ayant le rôle magasin de se connecter, de compléter son profil, d'accéder au catalogue, de rechercher des fournisseurs ou des produits, puis de consulter les pages de détail disponibles. Le tableau de bord magasin centralise les actions importantes : accéder au catalogue, suivre les demandes envoyées, améliorer le profil et retrouver les éléments utiles à son activité de sourcing.

Le catalogue est l'écran central de ce parcours. Il permet au magasin d'identifier rapidement une offre pertinente, d'analyser les informations d'un produit ou d'un fournisseur, puis d'envoyer une demande structurée de contact ou de devis. Cette demande représente l'action principale du parcours magasin, car elle transforme la recherche en opportunité commerciale concrète pour le fournisseur.

3.4 Fonctionnalités principales livrées

Le MVP livré couvre les fonctionnalités essentielles définies dans le périmètre initial. Il inclut l'authentification, la gestion des rôles, les profils fournisseurs, les profils magasins, la gestion des produits, le catalogue, les pages de détail, les demandes de contact ou de devis, ainsi que les tableaux de bord associés aux deux rôles.

Le système distingue clairement les utilisateurs fournisseurs et magasins. Cette séparation permet de proposer des interfaces adaptées à chaque besoin. Le fournisseur agit principalement sur sa visibilité et son catalogue. Le magasin agit principalement sur la recherche, l'analyse des offres et l'envoi de demandes.

Le backend expose les routes nécessaires à ces fonctionnalités sous forme d'API REST. Il est organisé en modules séparés : authentification, utilisateurs, fournisseurs, magasins, produits, catégories, demandes et santé de l'application. Le frontend consomme ces données via des services dédiés et affiche les pages correspondantes dans une interface React structurée.

3.5 État de déploiement

À la clôture du Stage 5, KERNO est fonctionnel en local. Le projet peut être lancé sur un environnement de développement avec un backend Express, un frontend Vite et une base PostgreSQL locale. Les fichiers d'environnement d'exemple permettent de documenter les variables nécessaires au lancement.

Le projet n'est pas encore en production. Cette limite est importante à préciser dans le rapport final et lors de la présentation. Le MVP est terminé pour le cadre du Portfolio Project, mais il n'est pas encore prêt à être exploité commercialement sans une phase complémentaire de déploiement et de durcissement production.

La prochaine étape technique sera donc de préparer une version hébergée, avec une base de données distante, une configuration sécurisée, des variables d'environnement adaptées, une politique CORS de production, un domaine, un monitoring minimal et une vérification complète des parcours en conditions réelles.



04

Comparaison entre
les objectifs initiaux et les
résultats finaux

4. Comparaison entre les objectifs initiaux et les résultats finaux

4.1 Objectif de la section

Cette section compare les objectifs définis pendant la phase de cadrage avec l'état réel du MVP livré à la clôture du Stage 5. L'objectif est de vérifier si le projet final respecte le périmètre prévu, d'identifier les écarts éventuels et d'expliquer les arbitrages effectués pendant le développement.

Le Stage 3 avait défini KERNO comme une marketplace B2B SaaS centrée sur une chaîne simple : un fournisseur crée son profil, ajoute ses produits, un magasin recherche une offre, consulte les informations disponibles, puis envoie une demande de contact ou de devis. À la fin du Stage 5, le MVP livré reste aligné avec cet objectif.

4.2 Comparaison fonctionnelle globale

L'objectif fonctionnel initial était de couvrir deux parcours principaux : le parcours fournisseur et le parcours magasin. Le fournisseur devait pouvoir créer sa présence sur la plateforme, publier des produits et consulter les demandes reçues. Le magasin devait pouvoir rechercher des offres, consulter les informations disponibles et envoyer une demande structurée.

Cet objectif a été atteint pour le périmètre MVP. Le produit final permet de démontrer le parcours central prévu : fournisseur, profil, produits, catalogue, magasin, demande, consultation fournisseur. La logique produit reste donc cohérente avec la proposition de valeur initiale.

Le projet ne couvre pas encore des fonctionnalités avancées comme la transaction, le paiement, la livraison ou la messagerie complète. Cet écart n'est pas un retard par rapport au MVP : ces fonctionnalités avaient été volontairement classées hors périmètre afin d'éviter une dérive fonctionnelle.

4.3 Comparaison technique

Les choix techniques prévus au Stage 3 ont été conservés dans le projet final. Le frontend repose sur React, JavaScript et Vite. Le backend repose sur Node.js, Express et JavaScript. La base de données est structurée avec PostgreSQL et Prisma ORM. L'API suit une logique REST et dispose d'une documentation Swagger.

Le résultat final confirme également le choix d'un backend monolithe modulaire. Le repo contient des modules séparés pour l'authentification, les utilisateurs, les fournisseurs, les magasins, les produits, les catégories, les demandes et la santé de l'application. Cette organisation garde une seule API backend tout en isolant les responsabilités fonctionnelles.

La base de données correspond aussi au périmètre prévu. Les données principales du MVP concernent les utilisateurs, les profils fournisseurs, les profils magasins, les produits, les catégories et les demandes de contact ou de devis. Cette structure relationnelle est cohérente avec les flux du produit et reste suffisamment simple pour une première version.

4.4 Comparaison organisationnelle

L'organisation initiale prévoyait une gestion du code source avec Git et GitHub, des branches, des commits clairs, des pull requests et une stratégie qualité adaptée au contexte du MVP.

Dans la pratique, cette organisation a été renforcée par l'utilisation de GitHub Project. Lors du cadrage initial, l'équipe avait envisagé Trello pour le suivi des tâches. Pendant le développement, ce choix a évolué vers GitHub Project, plus cohérent avec le dépôt, les issues, les branches et les pull requests. Ce changement a permis de centraliser le suivi projet et l'activité de développement dans le même environnement.

Le repo final contient également des documents de sprint, des liens de suivi, des documents de test, des preuves de validation et une documentation de revue. Les sprints ont été structurés autour de la fondation du projet, du backend, du frontend, de l'intégration, de la QA, de la documentation et de la préparation finale.

4.5 Écart principal restant : le déploiement

Le principal écart entre l'ambition future du projet et l'état final actuel concerne le déploiement. Le MVP est fonctionnel en local, mais il n'est pas encore disponible en production.

Cet écart est assumé. Le Stage 5 clôture le MVP académique, mais KERNO est destiné à être poursuivi comme projet entrepreneurial. Le déploiement, le durcissement sécurité, la configuration d'une base distante, la gestion des secrets, le monitoring, les sauvegardes et la validation en conditions réelles devront faire partie des prochaines étapes.

L'absence de déploiement public ne remet donc pas en cause la validation du MVP. Elle marque plutôt la transition entre une version de démonstration locale et une future version exploitable commercialement.

4.6 Synthèse de la comparaison

La comparaison entre les objectifs initiaux et le résultat final montre que KERNO respecte son périmètre MVP. Les fonctionnalités essentielles sont présentes, les exclusions ont été maintenues, les choix techniques sont cohérents avec la documentation initiale et l'organisation projet a été structurée autour de GitHub, des sprints, des tests et de la documentation.

Le projet final ne doit pas être présenté comme un produit commercial terminé, mais comme une base MVP complète et démontrable. Il valide la proposition de valeur principale : faciliter la mise en relation entre fournisseurs et magasins grâce à des profils, des produits, un catalogue et des demandes structurées.



05

Résultats finaux et livrables
produits

5. Résultats finaux et livrables produits

5.1 Objectif de la section

Cette section présente les résultats concrets obtenus à la fin du projet KERNO. Elle distingue les livrables fonctionnels, techniques, documentaires, organisationnels et qualité. L'objectif est de montrer que le projet ne se limite pas à une interface visible, mais qu'il repose sur un ensemble cohérent de fichiers, de parcours, de tests, de documentation et de supports de démonstration.

À la clôture du Stage 5, KERNO est livré comme un MVP fonctionnel en local. Il permet de démontrer les parcours principaux fournisseur et magasin, tout en respectant le périmètre défini initialement : profils, produits, catalogue, détails, demandes de contact ou de devis, suivi simple des demandes.

5.2 Livrable produit principal

Le livrable principal est une application web MVP nommée KERNO. Elle prend la forme d'une plateforme B2B SaaS destinée à faciliter la mise en relation entre fournisseurs directs ou locaux et magasins de détail.

Le produit livré comprend une interface publique, un système d'authentification, deux rôles utilisateurs, des espaces dédiés fournisseur et magasin, un catalogue, des pages de détail, des formulaires et un système de demandes. Le parcours utilisateur principal peut être démontré de bout en bout en environnement local.

Le MVP permet ainsi de montrer une première version exploitable de la proposition de valeur : un fournisseur peut créer sa présence, présenter ses produits et recevoir des demandes ; un magasin peut rechercher des offres, consulter les informations disponibles et initier un premier contact commercial.

5.3 Livrables frontend

Le frontend livré repose sur React, JavaScript et Vite. Il contient les pages nécessaires aux parcours publics, fournisseur et magasin. Les pages principales couvrent notamment la landing page, la connexion, l'inscription, les dashboards, les profils, le catalogue, les pages de détail produit et fournisseur, le formulaire de demande, les demandes envoyées, les demandes reçues et les détails de demande.

L'organisation du frontend repose sur une séparation par responsabilités : composants, pages, layouts, routes, services, utilitaires, données statiques et styles. Cette structure permet de distinguer l'affichage, la navigation, les appels API, les composants réutilisables et les règles d'interface.

Le travail frontend comprend également une forte dimension visuelle. L'interface finale reprend une direction artistique plus professionnelle et cohérente avec le positionnement SaaS B2B de KERNO : pages publiques plus travaillées, tableaux de bord différenciés, cartes produits et fournisseurs, badges de statut, composants réutilisables, responsive, harmonisation typographique et amélioration des parcours.

5.4 Livrables backend

Le backend livré repose sur Node.js, Express et JavaScript. Il expose une API REST utilisée par le frontend et centralise la logique métier principale : authentification, gestion des rôles, profils fournisseurs, profils magasins, produits, catégories, demandes et santé de l'application.

Le backend est structuré en monolithe modulaire. Les domaines fonctionnels sont séparés en modules : auth, users, suppliers, stores, products, categories, requests et health. Chaque module contient ses routes, contrôleurs, services, documentation Swagger et README lorsque cela est pertinent. Cette organisation rend le backend plus lisible et plus facile à expliquer lors d'une revue technique.

Le backend contient aussi les middlewares nécessaires au fonctionnement global de l'application : authentification, gestion des erreurs et gestion des routes non trouvées. Cette structure permet de centraliser les contrôles transverses et de limiter la duplication dans les modules.

5.5 Livrables base de données

La base de données livrée repose sur PostgreSQL et Prisma ORM. Le dossier Prisma contient le schéma de données, les migrations, le fichier de verrouillage de migration, le seed et une documentation d'installation locale.

Le modèle de données couvre les entités essentielles du MVP : utilisateurs, profils fournisseurs, profils magasins, catégories, produits et demandes de contact. Cette structure correspond au périmètre initial du projet et permet de représenter les interactions principales entre les deux rôles utilisateurs.

Le repo contient plusieurs migrations Prisma identifiées dans l'historique du projet. Elles montrent l'évolution progressive du schéma pendant le développement : création du schéma MVP initial, alignement du statut des demandes, évolution des informations de prix produit et ajout d'une unité de prix supplémentaire.

5.6 Livrables API et Swagger

L'API REST livrée permet au frontend de communiquer avec le backend. Les principales opérations du MVP sont couvertes : inscription, connexion, récupération de l'utilisateur connecté, gestion des profils, gestion des produits, consultation du catalogue, consultation des détails, création et suivi des demandes.

La documentation Swagger est intégrée au backend. Elle permet de consulter et tester les routes principales de l'API. Ce livrable est important pour la revue technique, car il rend visibles les endpoints et facilite l'explication du lien entre frontend, backend et base de données.

L'API respecte la logique prévue dans le Stage 3 : chaque action principale du MVP correspond à une route backend et la documentation OpenAPI / Swagger sert de support de compréhension et de test.

5.7 Livrables tests, qualité et sécurité

Le projet contient plusieurs livrables de test et de validation. Le backend dispose d'un script de test et de tests API complets. Le frontend dispose d'une configuration Playwright pour la régression MVP. Le dossier documentation contient également des plans de test, des rapports de régression, des preuves de test et une collection Postman/Newman.

Le projet contient aussi des éléments liés à la sécurité. L'authentification repose sur une logique d'inscription, connexion, mot de passe hashé, jeton ou cookie d'authentification, routes protégées et vérification du rôle utilisateur.

La documentation sécurité couvre les décisions d'authentification, les protections d'accès API et l'état de l'audit de sécurité du MVP. Des scripts d'audit et de smoke tests sécurité sont aussi présents dans le dossier dédié. Ils matérialisent une démarche de vérification ciblée sur les comportements sensibles du backend : accès, ownership, configuration de production et robustesse des réponses d'erreur.

5.8 Livrables documentation

La documentation constitue un livrable important du projet. Le dossier docs centralise les documents liés au produit, à l'architecture, à l'API, à la base de données, à la sécurité, aux tests, à la démonstration, à la revue, aux audits, aux sprints et à Docker.

Cette documentation permet de comprendre comment le frontend React, le backend Express, le schéma Prisma et la base PostgreSQL fonctionnent ensemble. Elle facilite également l'onboarding d'un nouveau contributeur, la préparation de la revue manuelle et la présentation technique du projet.

Les principaux documents produits couvrent notamment l'architecture applicative, la structure backend, la structure frontend, l'architecture CSS, le schéma de base de données, le résumé API, les notes de sécurité, les preuves de test, le scénario de démonstration et les notes de revue technique.

5.9 Livrables organisationnels

Le projet a également produit des livrables d'organisation. Le dépôt GitHub centralise le code source, les branches, les pull requests, les issues et les documents associés. L'équipe a utilisé GitHub Project pour suivre le travail, en remplacement de Trello initialement envisagé.

Ce changement d'outil a permis de rapprocher le suivi projet de l'activité réelle de développement. Les tâches pouvaient être reliées plus facilement aux issues, aux branches, aux PR et aux évolutions du dépôt. Cette centralisation a rendu le suivi plus cohérent avec une logique de projet logiciel.

Le GitHub Project contient également des idées d'évolutions futures conservées pour les prochaines phases. Ces idées ne sont pas abandonnées : elles constituent un backlog produit destiné à faire évoluer KERNO après la validation du MVP. Ce choix permet de garder une trace organisée des pistes à explorer, sans les confondre avec les fonctionnalités nécessaires à la version actuelle.

5.10 Livrables de démonstration

Le projet comprend un scénario de démonstration destiné à guider la présentation finale. Ce livrable permet de structurer la démo autour du parcours principal du MVP : présentation de KERNO, connexion, parcours fournisseur, gestion des produits, parcours magasin, catalogue, pages de détail, envoi de demande et consultation côté fournisseur.

Le scénario de démo est essentiel car il limite le risque d'improvisation pendant la présentation. Il permet de montrer le produit dans un ordre logique, en restant concentré sur la valeur principale du MVP plutôt que sur des détails secondaires.

La démonstration doit être réalisée en local, puisque le projet n'est pas encore déployé en production. Un plan B avec captures d'écran ou vidéo courte devra être prévu pour sécuriser la présentation en cas de problème technique.

5.11 Métriques finales des livrables

Les livrables du projet peuvent être résumés par plusieurs indicateurs quantifiables. Ces métriques permettent de rendre le résultat final plus concret et de faciliter la lecture du niveau d'avancement. Elles sont regroupées ici afin d'éviter les répétitions dans le reste du rapport.

D'après l'arborescence fournie, le projet comprend deux parcours utilisateurs principaux, deux rôles utilisateurs, huit modules backend principaux, six entités principales liées au MVP en base de données, plusieurs migrations Prisma, dix-huit pages frontend principales, neuf familles de composants frontend, vingt-deux fichiers CSS structurés, dix dossiers principaux de documentation, une collection Postman/Newman, un test end-to-end Playwright MVP, un script de test backend complet et quatre scripts de sécurité ou d'audit.

Ces indicateurs montrent que KERNO dispose d'un socle complet pour un MVP : parcours fonctionnels, architecture frontend/backend, base relationnelle, API documentée, documentation, tests, sécurité, suivi projet et support de démonstration. Ces métriques ont été consolidées à partir du repository, des routes déclarées, des PR mergées et des résultats de tests disponibles au moment de la clôture du rapport.

5.12 Synthèse des résultats finaux

Le résultat final est conforme à l'objectif du Stage 5 : clôturer un MVP fonctionnel, documenté, démontrable et prêt pour une revue manuelle. KERNO permet de démontrer la mise en relation entre fournisseurs et magasins à travers des profils, des produits, un catalogue et des demandes structurées.

Le projet ne doit pas être présenté comme une marketplace commercialement finalisée, mais comme un MVP abouti. Il valide le cœur de valeur du produit et fournit une base technique solide pour une suite entrepreneuriale.

Les livrables produits, techniques, documentaires et organisationnels montrent que l'équipe a mené le projet au-delà d'une simple preuve de concept. Le MVP est stable pour une démonstration locale, documenté pour être compris, et suffisamment structuré pour être poursuivi après la formation.



06

Difficultés rencontrées et
solutions apportées

6. Difficultés rencontrées et solutions apportées

6.1 Objectif de la section

Cette section présente les principales difficultés rencontrées pendant le projet KERNO et les solutions mises en place pour les résoudre. L'objectif n'est pas de masquer les problèmes, mais de montrer comment l'équipe a su réagir, arbitrer et améliorer progressivement le MVP.

Le développement d'un MVP complet implique des choix techniques, fonctionnels et organisationnels. Certaines difficultés étaient liées au périmètre du projet, d'autres à l'intégration frontend/backend, à la qualité visuelle, aux tests, à la sécurité ou à la coordination de l'équipe.

6.2 Maîtrise du périmètre MVP

Une première difficulté a été de maintenir un périmètre réaliste. Le concept de KERNO pouvait naturellement évoluer vers une marketplace complète avec paiement, commandes, livraison, messagerie, favoris, avis, abonnements et analytics. Ces idées étaient intéressantes, mais trop ambitieuses pour une première version dans le cadre du Portfolio Project.

La solution a été de revenir régulièrement à la chaîne de valeur principale du MVP : fournisseur, profil, produits, catalogue, demande, consultation fournisseur. Ce fil conducteur a permis de distinguer les fonctionnalités indispensables des idées secondaires. Les fonctionnalités hors MVP ont été assumées comme des pistes futures, et non comme des manques du produit final.

Cette maîtrise du périmètre a été essentielle pour livrer un MVP cohérent plutôt qu'un produit trop large, instable ou incomplet.

6.3 Organisation du suivi projet

Au départ, l'utilisation de Trello avait été envisagée pour suivre les tâches. Pendant le projet, l'équipe a finalement utilisé GitHub Project, plus adapté au contexte du développement logiciel.

Ce changement a représenté une adaptation organisationnelle. L'avantage de GitHub Project était de centraliser le suivi dans le même environnement que le dépôt, les issues, les branches et les pull requests. Cela a permis de mieux relier les tâches aux évolutions concrètes du code.

La difficulté était de maintenir une discipline de suivi régulière. Un outil projet n'apporte de la valeur que s'il est mis à jour et utilisé de manière cohérente. Cette expérience a montré l'importance de garder les tâches simples, reliées au code et compréhensibles par toute l'équipe.

6.4 Intégration frontend/backend

L'intégration entre le frontend React et le backend Express a demandé une attention particulière. Le frontend devait consommer correctement l'API, gérer les états de chargement, les erreurs, les données utilisateur, les rôles et les redirections. Le backend devait de son côté exposer des routes cohérentes, protéger les accès et renvoyer des réponses exploitables.

Les difficultés principales concernaient la synchronisation entre les données attendues côté frontend et les données réellement fournies par l'API. Lorsqu'un champ, un statut ou une structure de réponse évoluait côté backend, il fallait adapter les services frontend et vérifier les pages concernées.

La solution a été de structurer les appels API dans des services dédiés, plutôt que de disperser les requêtes dans les composants. Cette organisation a rendu les corrections plus ciblées et a facilité la stabilisation des parcours.

6.5 Gestion des rôles et des accès

KERNO repose sur deux rôles principaux : fournisseur et magasin. Cette séparation est simple à comprendre côté produit, mais elle demande de la rigueur côté technique. Certaines pages doivent être accessibles uniquement au fournisseur, d'autres uniquement au magasin, et certaines actions doivent vérifier le propriétaire de la ressource.

La difficulté était d'éviter les incohérences d'accès : un magasin ne doit pas modifier les produits d'un fournisseur, un fournisseur ne doit pas accéder aux espaces réservés aux magasins, et les demandes doivent rester consultables par les bons utilisateurs.

La solution a été de renforcer les routes protégées, les contrôles de rôle et les vérifications côté backend. Côté frontend, les composants de protection de routes et la navigation adaptée au rôle ont permis de clarifier l'expérience utilisateur.

6.6 Qualité visuelle et cohérence UX

Une autre difficulté importante a été l'harmonisation visuelle du projet. Le MVP ne devait pas seulement fonctionner techniquement ; il devait aussi être présentable, cohérent et crédible pour une démonstration professionnelle.

Les pages publiques, les dashboards, les profils, le catalogue, les produits et les demandes devaient partager une même direction artistique. Il a donc fallu travailler les espacements, les titres, les cartes, les boutons, les couleurs, les statuts, les barres de section, les responsive fixes et la cohérence générale.

La solution a été de stabiliser progressivement un système visuel commun : composants UI réutilisables, styles dédiés par zone, harmonisation typographique, CTA cohérents et direction artistique SaaS B2B. Ce travail a permis d'obtenir un rendu plus professionnel et plus proche d'un produit réel.

6.7 Tests et stabilisation

Le projet a aussi demandé une phase de stabilisation. Une fonctionnalité peut fonctionner isolément, mais provoquer des effets de bord sur d'autres pages. Les modifications visuelles, les corrections de routes, les statuts de demandes, les accès par rôle ou les formulaires nécessitaient des vérifications régulières.

La difficulté était de ne pas se limiter à tester uniquement le scénario idéal. Il fallait aussi vérifier les cas d'erreur, les accès interdits, les données absentes, les statuts, les écrans vides et les comportements responsive.

La solution a été de combiner plusieurs types de validation : tests backend/API, tests Postman/Newman, test end-to-end Playwright, build frontend, lint, audits et vérifications manuelles. Cette approche n'équivaut pas encore à une couverture complète de production, mais elle est cohérente avec le niveau attendu pour un MVP.

6.8 Sécurité et préparation production

La sécurité a représenté un point de vigilance important. Même si KERNO est encore local, le projet manipule des comptes utilisateurs, des rôles, des profils, des demandes et des données professionnelles. Il fallait donc éviter de traiter la sécurité comme un sujet secondaire.

Les difficultés concernaient principalement l'authentification, la protection des routes, les variables d'environnement, les secrets, les contrôles d'accès et la distinction entre environnement local et production.

La solution a été de documenter les éléments de sécurité, de prévoir des fichiers d'environnement d'exemple, d'utiliser des middlewares d'authentification, de séparer les accès par rôle et d'ajouter des notes d'audit sécurité. Le déploiement futur nécessitera toutefois un durcissement complémentaire : configuration production, HTTPS, CORS strict, secrets robustes, monitoring et sauvegardes.

6.9 Déploiement non réalisé à ce stade

Le principal point non réalisé à la clôture du Stage 5 est le déploiement en production. Le MVP est fonctionnel en local, mais il n'est pas encore accessible publiquement.

Cette limite ne remet pas en cause la validation du MVP académique, car le projet peut être lancé, testé et démontré localement. En revanche, elle constitue une étape importante pour la suite entrepreneuriale du projet.

La solution retenue est de traiter le déploiement comme une prochaine étape, après la clôture du Portfolio Project. Cette étape devra inclure l'hébergement frontend/backend, la base de données distante, la configuration des variables d'environnement, le durcissement sécurité, les tests post-déploiement et la préparation d'un environnement exploitable par de vrais utilisateurs.

6.10 Synthèse des difficultés

Les difficultés rencontrées pendant KERNO ont principalement concerné le périmètre, l'intégration, la qualité visuelle, les rôles, les tests, la documentation et la préparation production.

Ces difficultés ont été utiles car elles ont obligé l'équipe à faire des choix. Le projet a gagné en clarté en conservant un MVP centré sur la mise en relation. Il a gagné en solidité grâce à une architecture modulaire, une documentation structurée et plusieurs niveaux de validation. Il a gagné en crédibilité grâce au travail d'harmonisation UX/UI.

La principale amélioration future reste le passage d'un MVP local à un produit déployé, sécurisé et testé avec de vrais utilisateurs.

A large, stylized, light pink fox head is positioned in the background, facing left. It has a white circular eye and a white curved line for a mouth. The fox's body and tail are also in a light pink color, extending towards the bottom right of the frame.

07

Leçons apprises

7. Leçons apprises

7.1 Objectif de la section

Cette section présente les principaux enseignements tirés du projet KERNO. Elle complète la section précédente sur les difficultés rencontrées, mais avec un angle plus réflexif : ce que l'équipe a appris, ce qui devra être conservé dans de futurs projets, et ce qui devra être amélioré.

Le Stage 5 demande de documenter les résultats, les succès, les défis et les leçons apprises. L'objectif est donc de montrer que le projet a permis une progression technique, organisationnelle et méthodologique, au-delà de la simple livraison d'un MVP.

7.2 Le cadrage MVP est essentiel

La première leçon concerne l'importance du cadrage. KERNO aurait pu devenir rapidement trop large : marketplace complète, paiement, commandes, messagerie, livraison, analytics, abonnements, avis, favoris et automatisations. Ces fonctionnalités auraient ajouté de la valeur dans une version future, mais elles auraient aussi rendu le projet difficile à terminer dans les délais.

Le fait d'avoir défini un périmètre clair dès le Stage 3 a permis de garder une direction stable. Le MVP devait valider une chaîne simple : fournisseur, produits, recherche magasin, demande, réception fournisseur. Cette logique a servi de repère pendant tout le développement.

La leçon principale est qu'un MVP réussi n'est pas celui qui contient le plus de fonctionnalités. C'est celui qui valide clairement une valeur principale avec un périmètre maîtrisé.

7.3 Une documentation initiale aide, mais doit rester vivante

Le Stage 3 a fourni une base importante : user stories, maquettes, architecture, base de données, API, SCM/QA et justifications techniques. Cette documentation a permis de ne pas partir dans le développement sans vision claire.

Cependant, le projet a montré qu'une documentation initiale ne doit pas être figée. Certaines décisions ont évolué pendant le développement. L'interface finale a été améliorée par rapport aux premières maquettes, certains choix UX ont été simplifiés, et l'organisation projet a évolué vers GitHub Project plutôt que Trello.

La leçon est qu'une bonne documentation doit guider le projet, mais ne doit pas empêcher les ajustements nécessaires. Elle sert de cadre, pas de blocage. Pour un futur projet, il faudra continuer à documenter tôt, mais aussi prévoir des mises à jour régulières au fil des décisions réelles.

7.4 GitHub Project était plus adapté que Trello

L'équipe avait initialement envisagé Trello pour suivre les tâches. En pratique, GitHub Project s'est révélé plus cohérent avec le fonctionnement du projet, car le code, les issues, les branches, les pull requests et le tableau de suivi restaient dans le même écosystème.

Ce choix a renforcé la traçabilité. Une tâche pouvait plus facilement être reliée à une issue, une branche ou une pull request. Cela a aussi rendu le suivi plus proche de la réalité du développement.

La leçon est qu'un outil d'organisation doit être choisi en fonction du contexte réel du projet. Pour une équipe qui développe sur GitHub, GitHub Project limite la dispersion et rend le suivi plus naturel qu'un outil externe.

7.5 Le monolithe modulaire est adapté à un MVP

Le choix d'un backend en monolithe modulaire s'est confirmé pendant le développement. Il a permis de garder une seule API, tout en organisant le code par domaines : auth, users, suppliers, stores, products, categories, requests et health.

Cette structure a facilité la compréhension du projet et la préparation à la revue technique. Elle évite la complexité prématurée des microservices tout en gardant un code lisible.

La leçon est qu'une architecture simple mais bien séparée vaut mieux qu'une architecture ambitieuse mais lourde à maintenir. Pour un MVP, la priorité doit rester la clarté, la rapidité de développement, la testabilité et la facilité d'explication.

7.6 L'intégration doit être testée en continu

Le projet a montré que les problèmes ne viennent pas seulement des fonctionnalités isolées. Une route API peut fonctionner seule, une page frontend peut s'afficher seule, mais l'intégration complète peut révéler des incohérences : noms de champs, formats de réponse, statuts, droits d'accès, redirections ou cas d'erreur.

La leçon est qu'il faut tester l'intégration régulièrement, et pas uniquement à la fin. Les services frontend, les routes backend et le modèle Prisma doivent rester synchronisés. Chaque changement côté API doit être vérifié côté interface, et chaque besoin frontend doit être validé côté backend.

Pour les futurs projets, il faudra prévoir plus tôt des scénarios de test bout en bout, même simples. Un parcours MVP automatisé ou semi-automatisé permet de détecter plus vite les régressions.

7.7 La qualité visuelle prend du temps

Une autre leçon importante concerne l'UI/UX. Construire une interface fonctionnelle est une chose. Obtenir une interface cohérente, professionnelle, responsive et présentable en démonstration demande beaucoup plus de temps.

KERNO a nécessité un travail important sur les pages publiques, les dashboards, les cartes, les boutons, les statuts, les alignements, la typographie, les espacements et l'harmonisation visuelle. Ce travail est souvent sous-estimé, alors qu'il influence fortement la perception du projet.

La leçon est qu'il faut intégrer la qualité visuelle dans le planning, et non la traiter comme une finition rapide. Pour un produit présenté à des évaluateurs ou à de futurs utilisateurs, l'UX fait partie du livrable.

7.8 La sécurité doit être anticipée, même sur un MVP local

Même si KERNO n'est pas encore déployé en production, la sécurité a dû être prise en compte. L'application gère des comptes, des rôles, des profils, des produits et des demandes. Il fallait donc protéger les routes sensibles, vérifier les rôles, éviter les secrets dans le repo et documenter les limites du MVP.

La leçon est qu'un MVP n'a pas besoin d'être parfait en sécurité, mais il doit éviter les erreurs critiques et identifier clairement ce qui devra être renforcé avant production : HTTPS, cookies, CORS strict, secrets robustes, monitoring, sauvegardes et tests sécurité post-déploiement.

7.9 Le déploiement doit être prévu plus tôt

Le MVP final est fonctionnel en local, mais il n'est pas encore déployé en production. Ce choix reste acceptable pour la clôture du Portfolio Project, mais il montre aussi qu'un déploiement ne doit pas être traité comme une simple étape finale.

La mise en production réelle nécessite plus que le code : configuration serveur, variables d'environnement, base distante, sécurité, CORS, domaine, monitoring, sauvegardes et tests post-déploiement.

La leçon est que le déploiement doit être anticipé dès le développement si l'objectif est de rendre le produit accessible publiquement. Pour la suite entrepreneuriale de KERNO, ce sera une priorité.

7.10 Synthèse des leçons apprises

Le projet KERNO a permis de tirer plusieurs enseignements majeurs. Le cadrage MVP est indispensable pour éviter la dérive fonctionnelle. Une documentation initiale est utile, mais elle doit rester évolutive. GitHub Project est plus adapté qu'un outil externe lorsque le développement se fait déjà sur GitHub. Une architecture simple, comme le monolithe modulaire, est pertinente pour un MVP.

Le projet a aussi montré que l'intégration, la qualité visuelle, la sécurité, la documentation et la préparation de la démo demandent autant de rigueur que le développement des fonctionnalités principales. Enfin, le déploiement devra être anticipé plus tôt dans la prochaine phase, car il représente une étape technique complète et non une simple formalité.



08

Rétrospective équipe

8. Rétrospective équipe

8.1 Objectif de la section

Cette section synthétise la rétrospective d'équipe demandée dans le Stage 5. L'objectif est d'évaluer la collaboration, les points forts, les difficultés rencontrées et les axes d'amélioration pour de futurs projets.

La rétrospective de KERNO doit être lue comme un bilan constructif. Le projet a été mené par une équipe de trois personnes, avec des responsabilités principales réparties entre frontend, backend, intégration, tests, documentation et préparation à la revue. Cette répartition a permis de faire avancer le MVP tout en gardant une vision commune du produit.

8.2 Organisation de l'équipe

L'équipe était composée de trois membres : Antoine Gousset, Gwendal Boisard et Yonas Houriez. Les responsabilités principales ont été réparties selon les besoins du projet et les compétences mobilisées pendant le développement.

Antoine Gousset a principalement travaillé sur le frontend, l'UI/UX, la documentation, la QA visuelle et la préparation de la démonstration. Gwendal Boisard a principalement porté le backend, l'API, la base de données, Prisma et la validation API. Yonas Houriez a contribué à l'intégration, au workflow GitHub, au support de test et à la préparation de la revue.

Cette organisation a permis de séparer les responsabilités sans fragmenter complètement le projet. Chaque membre avait un périmètre principal, mais l'objectif commun restait la livraison d'un MVP fonctionnel et démontrable.

8.3 Ce qui a bien fonctionné

Le premier point positif est la clarification progressive du périmètre. Le projet aurait pu devenir trop large, mais l'équipe a réussi à rester centrée sur la chaîne de valeur principale : fournisseur, produits, recherche magasin, demande et réception fournisseur. Cette discipline a permis de garder un MVP cohérent et de ne pas disperser les efforts.

Le deuxième point positif est la structuration technique du projet. Le backend en monolithe modulaire, le frontend organisé par pages, composants et services, ainsi que la documentation thématique ont facilité la compréhension globale du repo. Le projet est lisible, démontrable et explicable lors d'une revue technique.

Le troisième point positif est l'utilisation de GitHub comme environnement central. Le repo, les issues, les branches, les pull requests et GitHub Project ont permis de rapprocher le suivi projet du développement réel. Ce choix a remplacé Trello, qui était initialement envisagé, mais GitHub Project s'est révélé plus adapté au contexte du projet.

Le quatrième point positif est la progression sur la qualité. Le projet ne s'est pas limité à un parcours fonctionnel minimum. Il a aussi intégré des tests, de la documentation, des audits, une préparation de revue, un scénario de démonstration et une harmonisation visuelle importante.

8.4 Difficultés rencontrées en équipe

La première difficulté a été de maintenir un périmètre commun. KERN0 avait un potentiel large, avec de nombreuses idées d'évolution. Il a donc fallu arbitrer régulièrement entre ce qui était utile pour le MVP et ce qui devait rester hors périmètre.

La deuxième difficulté a été de synchroniser les différentes couches du projet. Le backend, le frontend, la base de données, les services API, les routes protégées et les données de démonstration devaient rester cohérents. Une modification dans une couche pouvait avoir des conséquences sur une autre.

La troisième difficulté a concerné le suivi des tâches. Un outil comme GitHub Project est efficace seulement si les issues restent suffisamment petites, claires et reliées à des objectifs concrets.

La quatrième difficulté a été la gestion du temps en fin de projet. La stabilisation, la QA visuelle, la documentation, les tests et la préparation de la démonstration demandent beaucoup de temps. Ces tâches peuvent être sous-estimées parce qu'elles arrivent après le développement visible des fonctionnalités.

8.5 Axes d'amélioration pour de futurs projets

Pour un futur projet, le premier axe d'amélioration serait de découper encore plus tôt les tâches. Des issues plus petites facilitent la répartition, la revue, les tests et la validation. Elles réduisent aussi le risque de pull requests trop larges ou difficiles à relire.

Le deuxième axe serait d'anticiper davantage l'intégration. Même si chaque partie fonctionne séparément, le vrai risque apparaît souvent au moment où le frontend, le backend et la base de données doivent fonctionner ensemble. Des tests d'intégration plus précoces permettraient de détecter plus vite les incohérences.

Le troisième axe serait de prévoir plus tôt le déploiement. Le MVP est fonctionnel en local, mais le passage en production représente un chantier complet. Pour un projet entrepreneurial, il faudra intégrer cette étape plus tôt dans la roadmap.

Le quatrième axe serait de formaliser davantage les critères de validation avant chaque merge. Cela permettrait de mieux sécuriser la qualité, le responsive, les droits d'accès, les erreurs API et les parcours critiques.

Le cinquième axe serait de programmer des points d'équipe courts mais réguliers. Même lorsque les responsabilités sont bien réparties, une synchronisation fréquente évite les écarts de compréhension et limite les corrections tardives.

8.6 Bilan de collaboration

La collaboration a permis de transformer une idée produit en MVP fonctionnel. L'équipe a su passer d'un cadrage documentaire à une application locale démontrable, avec une architecture réelle, des parcours utilisateurs, une base de données, une API, des tests, de la documentation et un support de présentation.

Le projet a aussi montré que la collaboration ne repose pas seulement sur la répartition des tâches. Elle demande une vision commune, des arbitrages partagés, un suivi clair, une capacité à corriger les écarts et une compréhension globale du produit par chaque membre.

L'équipe a progressé dans sa capacité à travailler sur un projet complet : cadrage, développement, intégration, qualité, documentation et préparation de revue. Les points à améliorer concernent surtout l'anticipation du déploiement, le découpage des issues et l'intégration plus précoce des tests.



09

Conclusion et
recommandations futures

9. Conclusion et recommandations futures

9.1 Objectif de la section

Cette section clôture le rapport final du Stage 5. Elle synthétise les résultats du projet KERNO, rappelle l'état du MVP livré, identifie les limites assumées et présente les recommandations pour la suite du projet.

9.2 Bilan général du projet

KERNO a permis de transformer une idée initiale en MVP complet. Le projet partait d'un besoin clair : faciliter la mise en relation entre fournisseurs directs ou locaux et magasins de détail. À la fin du Stage 5, cette proposition de valeur est démontrable à travers deux parcours principaux : le parcours fournisseur et le parcours magasin.

Le fournisseur peut créer son profil, présenter son activité, publier des produits et consulter les demandes reçues. Le magasin peut accéder au catalogue, rechercher des offres, consulter les pages de détail et envoyer une demande de contact ou de devis. Cette chaîne correspond au cœur du MVP défini lors des étapes précédentes.

Le produit livré reste volontairement limité. Il ne couvre pas le paiement, la commande complète, la livraison, la facturation, la messagerie avancée ou les analytics détaillés. Ces exclusions ne sont pas des échecs : elles correspondent à des arbitrages nécessaires pour conserver un MVP clair, réaliste et démontrable.

9.3 Résultat final

Le résultat final est un MVP fonctionnel en local, structuré autour d'un frontend React, d'un backend Express, d'une base PostgreSQL avec Prisma ORM et d'une API REST documentée avec Swagger.

Le projet dispose aussi d'une documentation importante : architecture, API, base de données, sécurité, tests, revue technique, audits, sprints et scénario de démonstration. Cette documentation renforce la lisibilité du projet et facilite sa présentation lors de la Manual QA Review.

L'équipe a également mis en place une organisation de travail cohérente avec un projet logiciel : dépôt GitHub, branches, issues, pull requests et GitHub Project. L'utilisation de GitHub Project a remplacé Trello, initialement envisagé, afin de centraliser le suivi projet dans le même environnement que le code.

9.4 Limites assumées

La principale limite du projet à la clôture du Stage 5 est l'absence de déploiement en production. KERN0 est prêt pour une démonstration locale, mais il n'est pas encore accessible publiquement. Cette limite est assumée, car le projet doit d'abord clôturer son périmètre MVP avant de passer à une phase de production.

Une autre limite concerne la validation utilisateur réelle. Le MVP démontre techniquement et fonctionnellement la proposition de valeur, mais il devra être testé avec de vrais fournisseurs et magasins pour confirmer l'intérêt marché, les besoins prioritaires, les freins d'usage et les opportunités commerciales.

Enfin, certaines fonctionnalités futures restent à arbitrer. Le projet pourrait évoluer vers une marketplace plus complète, mais cette évolution devra être progressive. Ajouter trop vite le paiement, les commandes ou une messagerie complète risquerait de complexifier le produit avant d'avoir validé les usages réels.

9.5 Recommandations techniques

La première recommandation est de préparer un déploiement propre. Cette étape devra inclure un hébergement frontend, un hébergement backend, une base PostgreSQL distante, une gestion stricte des variables d'environnement, une politique CORS de production, HTTPS, des secrets robustes et une stratégie de sauvegarde.

La deuxième recommandation est de renforcer la sécurité avant toute mise en ligne. Les contrôles de rôles, l'authentification, les cookies, les permissions, les erreurs API, les accès aux ressources et les données sensibles devront être revérifiés dans un contexte production.

La troisième recommandation est d'élargir les tests. Le MVP dispose déjà de tests et de validations, mais une version déployée nécessitera des tests plus complets : régression end-to-end, tests d'intégration, tests sur données réelles, vérification responsive, tests sécurité et tests de charge basiques.

La quatrième recommandation est de continuer à documenter les décisions. Si le projet évolue, la documentation devra rester à jour : architecture, schéma de base de données, routes API, règles métier, configuration de déploiement et procédures de maintenance.

9.6 Recommandations produit

La première recommandation produit est de tester KERNO auprès de vrais utilisateurs. Avant d'ajouter de nombreuses fonctionnalités, il faudra vérifier si les fournisseurs et les magasins comprennent rapidement la valeur de la plateforme, si le parcours est fluide et si la demande de contact répond à un vrai besoin.

La deuxième recommandation est de prioriser les améliorations selon les retours terrain. Les prochaines fonctionnalités ne devront pas être choisies uniquement parce qu'elles sont intéressantes techniquement, mais parce qu'elles renforcent la valeur réelle du produit.

Le GitHub Project contient déjà plusieurs idées d'évolutions futures qui n'ont pas été intégrées au MVP afin de préserver le périmètre initial. Ces idées ne sont pas abandonnées : elles sont conservées comme backlog produit pour les prochaines phases de KERNO. Elles permettront de faire évoluer progressivement la plateforme après la validation du MVP, en fonction des retours utilisateurs, des contraintes techniques et du potentiel entrepreneurial du projet.

Parmi les pistes conservées dans GitHub Project, on retrouve notamment des améliorations de performance avec des audits Lighthouse, du durcissement sécurité post-MR, l'étude d'une bibliothèque de composants React/UI, des inspirations de branding et de polish visuel, une possible intégration cartographique avec MapLibre GL JS, ainsi que des refactors techniques ciblés comme la pagination, le nettoyage de dépendances ou l'amélioration progressive de la gestion des tokens. Ces éléments restent volontairement dans le backlog afin de ne pas mélanger la validation du MVP avec les évolutions post-MVP.

Cette approche permet de distinguer clairement deux niveaux : d'un côté, le MVP livré dans le cadre du Portfolio Project ; de l'autre, les futures améliorations destinées à transformer KERNO en produit plus complet. Le fait de conserver ces idées dans GitHub Project permet de garder une trace organisée des pistes à explorer sans les confondre avec les fonctionnalités nécessaires à la validation actuelle.

La troisième recommandation est de travailler progressivement le modèle économique. Plusieurs pistes existent : abonnement fournisseur, mise en avant, visibilité premium, accès à des statistiques, offres sponsorisées ou service d'accompagnement. Ces pistes devront être testées avant d'être intégrées durablement.

La quatrième recommandation est de conserver la simplicité du MVP. KERNO doit d'abord rester efficace sur son cœur de valeur : rendre les fournisseurs visibles et aider les magasins à initier un premier contact qualifié.

9.7 Prochaines étapes recommandées

La prochaine étape immédiate est de finaliser la clôture académique du projet : rapport final, slide deck, démonstration live et demande de Manual QA Review.

Après cette clôture, les étapes recommandées sont les suivantes : préparer le déploiement, sécuriser la configuration production, tester les parcours sur une version hébergée, collecter des retours utilisateurs, corriger les frictions prioritaires, puis seulement ensuite envisager les fonctionnalités avancées.

Les idées déjà identifiées dans GitHub Project serviront de base au futur backlog produit, mais elles devront être priorisées après les premiers retours utilisateurs afin d'éviter d'ajouter des fonctionnalités avant d'avoir validé les besoins réels.

Les évolutions possibles incluent une messagerie simple, un suivi plus précis des demandes, des statistiques fournisseur, une gestion plus avancée des profils, une recherche améliorée, des favoris, des notifications, puis éventuellement un modèle de monétisation.

9.8 Conclusion finale

Le projet KERNO atteint les objectifs attendus pour le Stage 5. Le MVP est terminé pour son périmètre défini, les parcours principaux sont démontrables en local, les choix techniques sont cohérents, la documentation est structurée et les limites du produit sont clairement identifiées.

Le projet a permis à l'équipe de travailler sur un cycle complet : cadrage, conception, architecture, développement, intégration, tests, documentation, stabilisation et préparation de présentation. Il a également permis de tirer des leçons concrètes sur la gestion d'un MVP, la collaboration, la qualité, la sécurité et l'importance du périmètre.

KERNO constitue donc une base solide. Le projet est prêt à être présenté comme MVP Portfolio Project, puis à être poursuivi dans une logique entrepreneuriale avec une prochaine priorité claire : passer d'une démonstration locale à une version déployée, sécurisée et testée auprès de vrais utilisateurs.